

M.Tech CSE I Year II Semester

Advanced Computer Architecture

*An Exhaustive, Foundations-First Textbook on Parallelism, Scalability, and
Hardware Design*

Department of Computer Science and Engineering

Marri Laxman Reddy Institute of Technology and Management
(UGC Autonomous)

Course Code: PC - IV

Contents

Preface	4
1 Theory of Parallelism and Parallel Models	5
1.1 Introduction for Beginners: What is Computer Architecture?	5
1.1.1 The Physical Boundaries of Computation	6
1.2 The Memory Hierarchy Pyramid	6
1.3 Flynn’s Taxonomy	7
1.3.1 Single Instruction, Single Data (SISD)	7
1.3.2 Single Instruction, Multiple Data (SIMD)	7
1.3.3 Multiple Instruction, Single Data (MISD)	7
1.3.4 Multiple Instruction, Multiple Data (MIMD)	8
1.4 Shared-Memory Multiprocessors (UMA/NUMA)	8
1.4.1 Uniform Memory Access (UMA)	8
1.4.2 Non-Uniform Memory Access (NUMA)	8
1.5 Theoretical Models: PRAM and VLSI	8
2 Dependencies, Flow, and Interconnects	11
2.1 Bernstein’s Conditions	11
2.2 Data Dependencies and Program Graphs	12
2.3 Dataflow Execution Mechanisms	12
2.3.1 Activity Templates	13
2.4 System Interconnect Topologies	14
3 Scalability and Performance Scaling Laws	15
3.1 System Performance Metrics	15
3.2 Speedup Performance Laws	15
3.2.1 Amdahl’s Law (Fixed-Workload Scaling)	16
3.2.2 Gustafson’s Law (Scaled-Workload Scaling)	17
3.2.3 Visualizing Speedup Performance Curves	18

4	Pipelining Principles and Superscalar Design	19
4.1	A 5-Stage Linear Instruction Pipeline	19
4.1.1	Pipeline Space-Time Diagram	19
4.2	Non-Linear Pipeline Scheduling	20
4.2.1	Reservation Tables	20
4.2.2	Deriving Schedules and Collision Vectors	20
5	Shared-Memory Consistency and Coherence	22
5.1	Memory Consistency Models	22
5.1.1	Sequential Consistency (SC)	22
5.1.2	Weak Consistency (WC)	23
5.2	Cache Mapping Architectures	23
5.3	The MESI Cache Coherence Protocol	23
6	Vector and SIMD Architectures	25
6.1	Vector Processing Principles	25
6.1.1	Register-to-Register Vector Pipeline Execution Flow	25
6.2	The Connection Machine CM-5	26
	Conclusion	27

Preface

Computer architecture is the foundational bridge between the software we write and the physical hardware that executes it. In the early days of computing, this bridge was relatively simple: a single control unit fetched instructions one after another and executed them sequentially on a single processing element. However, as the physical limits of materials science and quantum mechanics have caught up with semiconductor scaling, the sequential computing model has reached its practical limits.

Today, performance scaling is driven almost entirely by parallelism, pipeline designs, and scalable architectures. This textbook is designed to introduce these advanced architectural paradigms. We start from absolute first principles, explaining abstract hardware logic using everyday analogies before introducing mathematical formulations and hardware block diagrams.

Every chapter is structured to build your intuition before introducing formal equations. Whether you are a first-year graduate student or an engineer looking to understand the physical reality of modern high-performance systems, this book will serve as a comprehensive, mathematically rigorous guide.

Chapter 1

Theory of Parallelism and Parallel Computer Models

To design computers that operate at high performance, we must first understand the fundamental physical and theoretical constraints that dictate how data moves through a system. This chapter establishes the evolutionary pathways of computing architectures, standardizes how we classify them, and analyzes the theoretical frameworks used to measure parallel processing limits.

1.1 Introduction for Beginners: What is Computer Architecture?

Before diving into the complex mathematics of parallel systems, let us establish what computer architecture actually represents.

A helpful analogy is to compare computer architecture to the design of a metropolitan transportation network:

- **Instruction Set Architecture (ISA):** This is the driver's manual. It specifies the rules of the road—what operations the vehicle can perform (addition, branch, load) and how the controls are labeled (registers).
- **Computer Organization (Microarchitecture):** This is what is under the hood of the vehicle. It represents the physical implementation of the ISA—how many cylinders the engine has (execution units), how wide the fuel injection pipes are (data buses), and how the fuel filter is structured (caches).

While a software engineer write programs using high-level languages, the microarchitecture is responsible for converting those instructions into physical electrical signals that charge capacitors, flip logic gates, and transfer data across copper and silicon wires.

1.1.1 The Physical Boundaries of Computation

For decades, microprocessors grew faster through two main principles:

1. **Moore's Law:** An empirical observation by Gordon Moore in 1965 stating that the number of transistors on a microchip doubles roughly every two years.
2. **Dennard Scaling:** A scaling formulation stating that as transistors shrink, their power density remains constant. This meant engineers could make transistors smaller, pack more of them together, and increase their clock speeds without increasing overall power dissipation.

By the mid-2000s, Dennard scaling reached its limit. As transistor gate oxide layers shrank to just a few atoms in thickness, quantum tunneling occurred, causing substantial electrical leakage. This leakage generated heat even when the transistor was idle. This physical limitation, known as the **Power Wall**, made it impossible to continue increasing clock frequencies without melting the physical silicon chip.

This boundary forced system designers to stop trying to build faster single-core processors and instead place multiple, moderately clocked processing elements on a single piece of silicon. This shift marked the beginning of modern parallel computer design.

1.2 The Memory Hierarchy Pyramid

In any computer system, there is a fundamental trade-off between speed, capacity, and cost:

- Memory that is extremely fast (such as registers) must be physically close to the execution units. Because of this proximity, it requires substantial physical space on the silicon die, which limits its capacity and makes it expensive.
- Memory that is highly capacious (such as magnetic hard drives or solid-state disks) is physically distant and complex to access, which makes it cheap but slow.

To balance these trade-offs, computer systems use a tiered **Memory Hierarchy**, which is traditionally represented as a pyramid.

The pyramid structure in Figure 1.1 functions through the principle of **Locality of Reference**:

- **Temporal Locality:** If a program accesses a memory location once, it is highly likely to access the exact same location again in the near future.
- **Spatial Locality:** If a program accesses a memory location, it is highly likely to access neighboring memory locations shortly thereafter (e.g., streaming through an array).

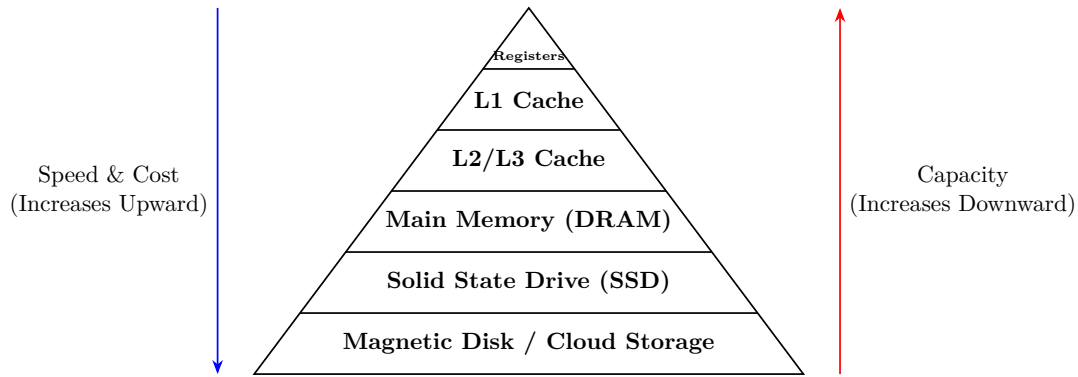


Figure 1.1: The typical Memory Hierarchy Pyramid showing speed, capacity, and cost trade-offs.

1.3 Flynn's Taxonomy

In 1966, Michael J. Flynn proposed a categorization system for computer architectures based on the flow of instruction streams and data streams through the hardware. Under Flynn's taxonomy, all architectures are classified into four distinct categories.

1.3.1 Single Instruction, Single Data (SISD)

This represents the traditional sequential computer model based on the classic Von Neumann architecture. A single control unit fetches one instruction at a time from memory. This instruction is decoded and executed by a single processing element on a single data stream.

1.3.2 Single Instruction, Multiple Data (SIMD)

In this architecture, a single, central control unit fetches and decodes a single instruction stream. However, instead of executing it on a single piece of data, the control unit broadcasts this instruction to multiple processing elements. Each processing element executes the operation on its own unique data element. This is the structural foundation of modern graphics processing units (GPUs) and vector processors.

1.3.3 Multiple Instruction, Single Data (MISD)

This structure involves multiple independent control units, each fetching different instructions, but executing them on the exact same data stream. This architecture is rarely used in general-purpose computing. Its primary application is in high-reliability, fault-tolerant control systems (such as spacecraft flight controllers or safety-critical railway systems), where multiple distinct algorithms process the same sensor data to cross-verify the results.

1.3.4 Multiple Instruction, Multiple Data (MIMD)

This represents the most flexible and widely used class of parallel computers. In a MIMD system, multiple independent processors run different programs (different instruction streams) on different datasets (different data streams). Modern multi-core processors, server clusters, and distributed systems are examples of MIMD architectures.

1.4 Shared-Memory Multiprocessors (UMA/NUMA)

Within MIMD systems, how physical memory is organized determines how processors communicate with one another.

1.4.1 Uniform Memory Access (UMA)

In a UMA system, all processors share a single, centralized physical memory via a system bus or crossbar interconnect. Every processor experiences the exact same access time (latency) to read or write to any memory location. Because of this uniform behavior, UMA architectures are also called Symmetric Multiprocessors (SMPs).

1.4.2 Non-Uniform Memory Access (NUMA)

As the number of processors scales, a centralized system bus becomes a performance bottleneck. To resolve this, NUMA architectures partition the physical memory and distribute it directly to individual processors.

Each processor can access its own local partition of memory with low latency. However, if a processor needs to access a memory address residing in another processor's partition (remote memory), it must route the request across an interconnect network, which incurs higher latency.

1.5 Theoretical Models: PRAM and VLSI

To analyze parallel algorithms independently of physical hardware constraints, researchers use the **Parallel Random Access Machine (PRAM)** model.

A PRAM model assumes:

1. P identical processors connected to a single, infinitely large, shared random-access memory.
2. All processors operate in lockstep, executing instructions in synchronized clock cycles.

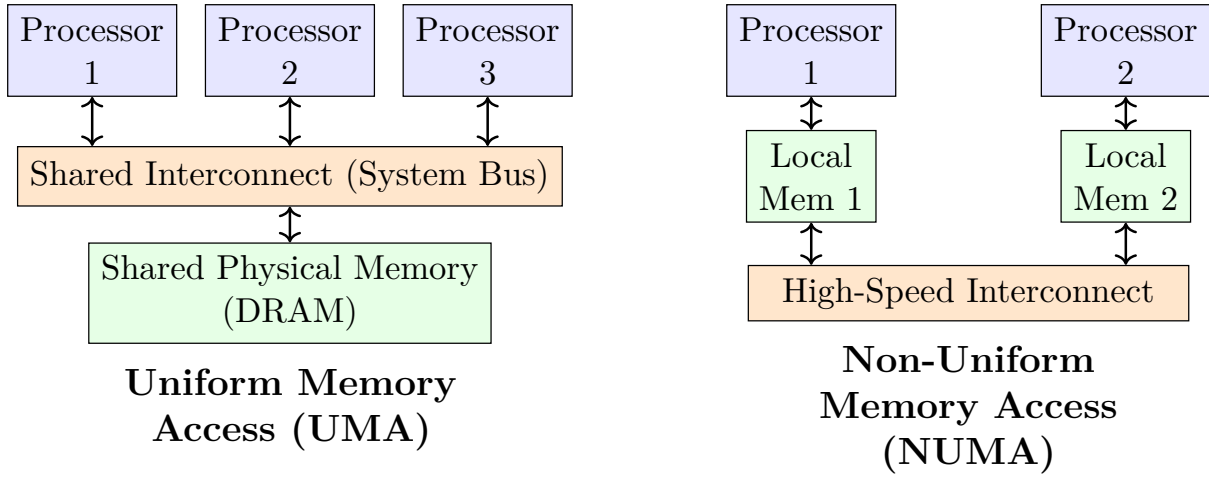


Figure 1.2: Physical architectural layouts contrasting UMA and NUMA systems.

3. Memory accesses are instantaneous and take exactly one clock cycle.

Based on how the model handles concurrent reads and writes to the same memory location, it is divided into several sub-models:

- **EREW (Exclusive Read, Exclusive Write):** No two processors can read or write to the same memory location at the same time. This is the most restrictive model.
- **CREW (Concurrent Read, Exclusive Write):** Multiple processors can read from the same memory location simultaneously, but only one processor can write to a location at any given instant.
- **CRCW (Concurrent Read, Concurrent Write):** Multiple processors can both read and write to the same location simultaneously. Write conflicts are resolved by specific strategies:
 - *Common:* All writing processors must write the exact same value; otherwise, an error occurs.
 - *Arbitrary:* One random processor's write succeeds, while others are discarded.
 - *Priority:* The processor with the lowest identification number (ID) has its write succeed.

While PRAM assumes ideal hardware, the **VLSI (Very Large Scale Integration) Complexity Model** accounts for physical constraints on silicon. It evaluates algorithms based on:

- **Area (A):** The physical amount of silicon space required to implement the circuit.
- **Time (T):** The number of clock cycles required to complete the computation.

Under the VLSI model, physical routing, wire length, and signal propagation delays are analyzed using metrics like the AT^2 trade-off.

Chapter 2

Program Dependencies, Flow Mechanisms, and Interconnects

Before parallelizing a program across multiple processing elements, we must analyze whether those elements can execute operations concurrently without corrupting the program's logic. This chapter details data dependency analysis, non-traditional execution flow paradigms, and physical interconnect network designs.

2.1 Bernstein's Conditions

To safely execute two instructions in parallel, we must guarantee that their execution order does not alter the final state of memory. This guarantee is governed by **Bernstein's Conditions**.

Let I_1 and I_2 be two instructions. We define:

- R_i : The set of memory locations or registers read by instruction I_i (the Read Set).
- W_i : The set of memory locations or registers written to by instruction I_i (the Write Set).

Definition 2.1 (Bernstein's Conditions). *Two instructions I_1 and I_2 can execute concurrently and yield deterministic results if and only if they satisfy the following three conditions:*

$$R_1 \cap W_2 = \emptyset \tag{2.1}$$

$$W_1 \cap R_2 = \emptyset \tag{2.2}$$

$$W_1 \cap W_2 = \emptyset \tag{2.3}$$

If any of these intersections are not empty, a dependency exists, and the instructions must be executed sequentially.

2.2 Data Dependencies and Program Graphs

Data dependencies represent hardware or software constraints that dictate instruction ordering:

1. **Read-After-Write (RAW) / Flow Dependency:** Instruction I_2 requires a value calculated by instruction I_1 . If I_2 runs before I_1 completes, it will read stale data. This is a true dependency.
2. **Write-After-Read (WAR) / Anti-Dependency:** Instruction I_2 writes to a register that instruction I_1 needs to read. If I_2 runs early, it will overwrite the value before I_1 can access it. This is a false dependency solvable by renaming registers.
3. **Write-After-Write (WAW) / Output Dependency:** Both I_1 and I_2 write to the exact same storage location. If I_2 completes before I_1 , the final value stored in the register will be the result of I_1 instead of I_2 . This is also a false dependency solvable by register renaming.

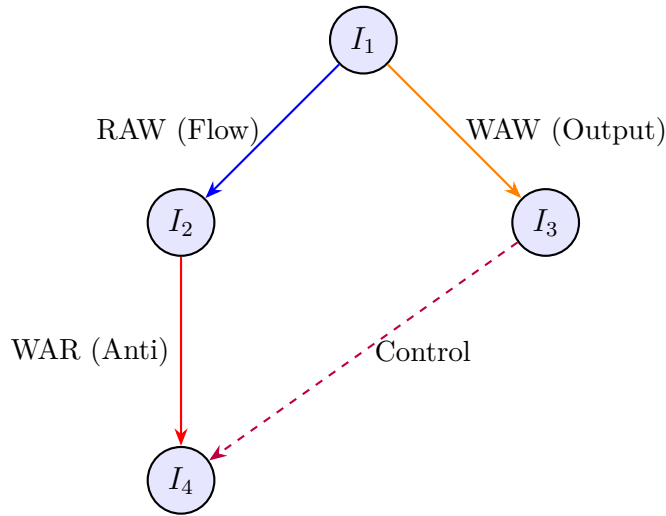


Figure 2.1: A Data Flow Program Graph (DFG) illustrating different classes of instruction dependencies.

2.3 Dataflow Execution Mechanisms

Traditional computers are governed by a **control-flow** mechanism: instructions are executed sequentially in the order they appear in the program, unless explicit control instructions (like jumps or branches) redirect execution.

An alternative paradigm is the **data-flow** mechanism, which has no program counter. An instruction is fetched and executed if and only if all of its input operands

become available. This model exposes the absolute maximum amount of instruction-level parallelism, though it requires complex tag-matching hardware to track operand availability.

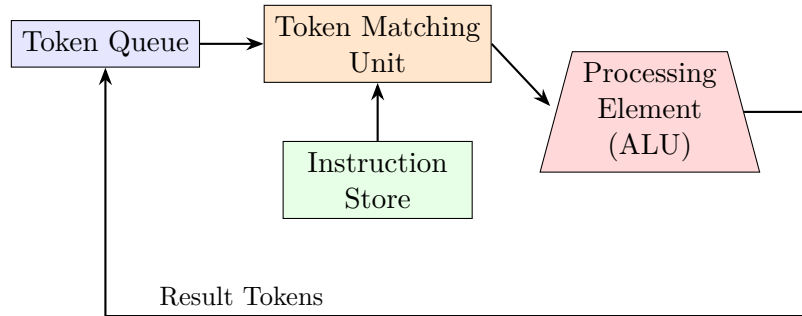


Figure 2.2: Block diagram of a Dataflow Execution Engine.

In a dataflow architecture:

- **Token Queue:** Stores data packets (tokens) containing operand values and destination instruction tags.
- **Matching Unit:** Matches incoming tokens with other operands required by the same instruction. Once all operands for an instruction arrive, they are packaged together.
- **Instruction Store / Node Store:** Holds the opcodes and templates for the program's instructions. . . .

2.3.1 Activity Templates

To implement this in hardware, instruction templates are stored as **Activity Templates**.

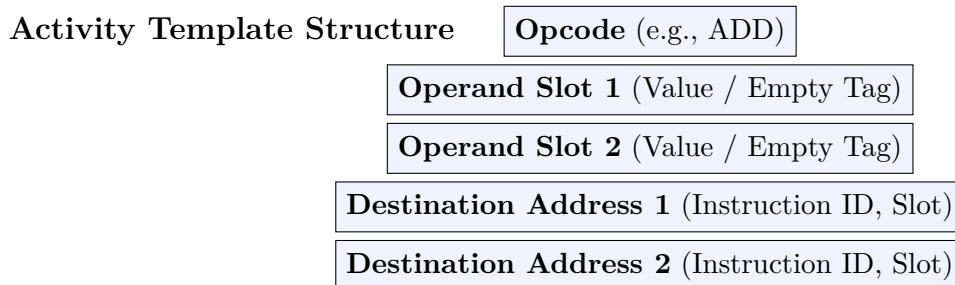


Figure 2.3: The structure of an Activity Template in a dynamic dataflow computer.

The template in Figure 2.3 remains idle in the instruction store until tokens arrive to fill its operand slots. Once both slots contain valid data, the hardware extracts the template, packages it as an executable packet, and routes it to the ALU.

2.4 System Interconnect Topologies

To coordinate processing nodes in MIMD systems, they must be physically connected by a communication network. We evaluate network topologies using three main metrics:

- **Node Degree (d):** The number of physical links connected to a single node. High node degree increases cost and wiring complexity but provides more direct communication paths.
- **Diameter (D):** The maximum shortest path distance between any two nodes in the network, measured in terms of physical hops. A smaller diameter means faster worst-case communication.
- **Bisection Bandwidth (B):** The minimum bandwidth available between two halves of the network when it is split into two equal-sized parts. High bisection bandwidth indicates that the network can handle heavy global traffic without bottlenecks.

Let us analyze three common topologies:

1. **2D Mesh (Figure 2.4):** Nodes are arranged in a 2D grid. It is highly suited for 2D silicon implementations (e.g., modern multi-core processors).
2. **Hypercube (Figure 2.5):** An n -dimensional hypercube connects 2^n nodes. Each node has degree n , and the maximum path diameter is also n . This logarithmic behavior provides low latency and high bisection bandwidth at the cost of complex high-dimensional physical wiring.
3. **Fat-Tree (Figure 2.6):** Unlike a traditional tree, where the links near the top are a major bottleneck, a Fat-Tree increases link bandwidth (or “thickness”) as you move up the hierarchy toward the root.

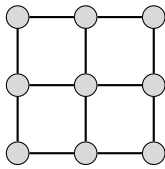


Figure 2.4: 2D Mesh.

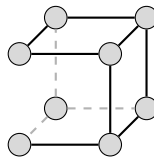


Figure 2.5: 3D Hypercube.

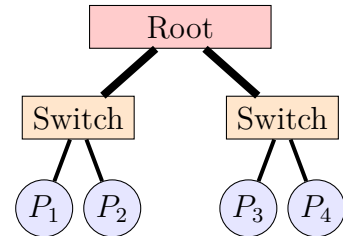


Figure 2.6: Fat-Tree.

Chapter 3

Scalability and Performance Scaling Laws

To build high-performance systems systematically, we must use quantitative metrics to measure how well our software scales as we add more processors. This chapter reviews system performance metrics and derives the fundamental laws of parallel speedup.

3.1 System Performance Metrics

We measure the execution performance of a processor using several standard variables:

- **Clock Cycle Time (t):** The duration of a single processor clock cycle. The inverse is the clock rate $f = 1/t$.
- **Cycles Per Instruction (CPI):** The average number of clock cycles required to execute a single instruction in a given program.
- **Instruction Count (I):** The total number of instructions in the compiled program.
- **MIPS (Millions of Instructions Per Second):** A historical metric representing execution speed, calculated as:

$$\text{MIPS} = \frac{I}{\text{Execution Time} \times 10^6} = \frac{f}{\text{CPI} \times 10^6} \quad (3.1)$$

3.2 Speedup Performance Laws

We define speedup (S_N) as the ratio of execution time on a single processor to the execution time on N processors:

$$S_N = \frac{T_1}{T_N} \quad (3.2)$$

How does speedup scale as we add more processors? This question is answered by two key scaling laws.

3.2.1 Amdahl's Law (Fixed-Workload Scaling)

Amdahl's Law, proposed by Gene Amdahl in 1967, models performance scaling under a rigid assumption: the workload to be solved is of fixed size.

Physical Analogy

Imagine a team of construction workers tasking themselves with building a single, specific backyard deck. Some tasks must be done sequentially (e.g., digging the post holes, waiting for concrete to cure). Other tasks can be done in parallel if you hire more workers (e.g., nailing down deck boards). No matter how many workers you hire, you cannot bypass the concrete curing time. The sequential part of the job acts as a hard limit on how fast the deck can be completed.

Mathematical Derivation of Amdahl's Law

Let T_1 be the total sequential execution time of a program on a single processor. Let s represent the fraction of the execution time that is inherently sequential and cannot be parallelized. Consequently, the fraction of the program that can be parallelized is $(1 - s)$.

When executed on N identical, fully cooperative parallel processors, the sequential portion still takes time $s \cdot T_1$, while the parallelizable portion scales down by a factor of N . The execution time on N processors, T_N , is:

$$T_N = s \cdot T_1 + \frac{(1 - s) \cdot T_1}{N} \quad (3.3)$$

Substituting this expression back into our definition of speedup ($S_N = T_1/T_N$):

$$S_N = \frac{T_1}{s \cdot T_1 + \frac{(1-s) \cdot T_1}{N}} \quad (3.4)$$

Factoring out T_1 from both the numerator and the denominator yields Amdahl's Law:

Theorem 3.1 (Amdahl's Law).

$$S_N = \frac{1}{s + \frac{1-s}{N}} \quad (3.5)$$

Lemma 3.1 (Asymptotic Limit of Amdahl's Law). *If we add an infinite number of processors to solve our fixed-size problem, we find the absolute upper limit of speedup:*

$$\lim_{N \rightarrow \infty} S_N = \lim_{N \rightarrow \infty} \frac{1}{s + \frac{1-s}{N}} = \frac{1}{s} \quad (3.6)$$

This limit reveals a harsh reality: if only 5% of a program is sequential ($s = 0.05$), the maximum speedup achievable is $1/0.05 = 20\times$, even if we deploy millions of processor cores.

3.2.2 Gustafson's Law (Scaled-Workload Scaling)

In 1988, John L. Gustafson pointed out a limitation of Amdahl's model: in practice, users do not keep their problem size fixed when they get access to larger supercomputers. Instead, they scale up the problem size to obtain more accurate or detailed results in the same amount of time.

Physical Analogy

Instead of trying to build the same backyard deck faster, a larger construction crew is given a fixed time frame (say, one week) and told to build the largest, most detailed structure possible. With more workers, the parallel work scales up, while the relative impact of sequential planning tasks diminishes.

Mathematical Derivation of Gustafson's Law

Let T_N represent the execution time of a scaled workload running on N processors. To simplify the math, we normalize this total parallel execution time to 1 unit of time:

$$T_N = s + p = 1 \quad (3.7)$$

where s is the sequential time fraction and p is the parallel execution fraction on the N -processor system. Thus, $p = 1 - s$.

If we were to run this same scaled workload on a single, isolated processor, the parallel portion would no longer be divided among N workers. It would take N times longer to complete. The total execution time on a single processor, T_1 , is:

$$T_1 = s + p \cdot N \quad (3.8)$$

Substituting $p = 1 - s$:

$$T_1 = s + (1 - s) \cdot N \quad (3.9)$$

Since we normalized the parallel execution time $T_N = 1$, the speedup S_N is:

$$S_N = \frac{T_1}{T_N} = \frac{s + (1 - s) \cdot N}{1} \quad (3.10)$$

This yields Gustafson's Law:

Theorem 3.2 (Gustafson’s Law).

$$S_N = s + (1 - s) \cdot N \quad (3.11)$$

This linear equation shows that speedup can scale continuously with the number of processors, provided the workload scales alongside the system size.

3.2.3 Visualizing Speedup Performance Curves

To observe how these two laws diverge, we plot their performance curves below.

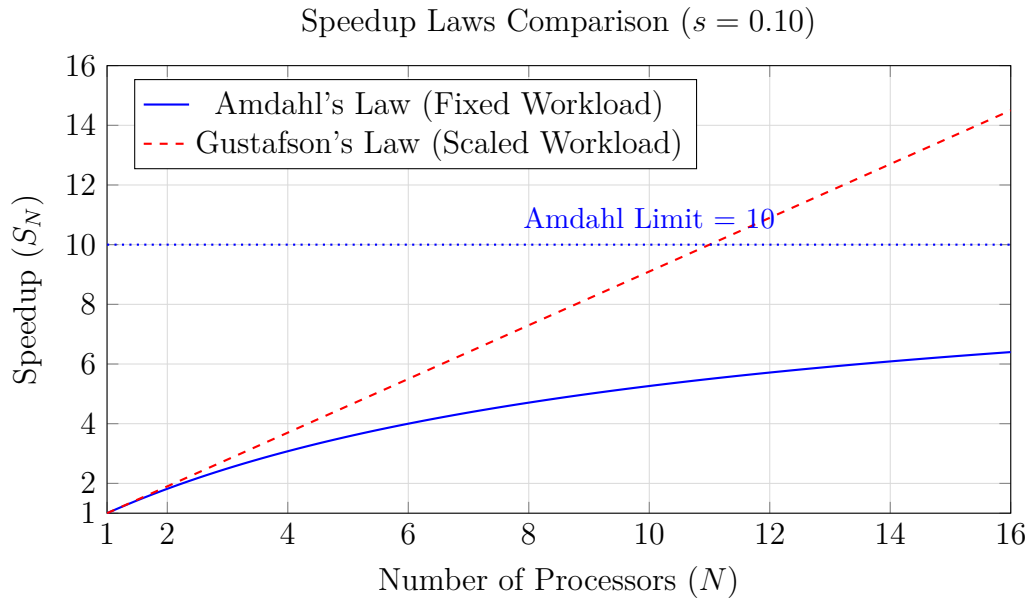


Figure 3.1: Mathematical comparison of Amdahl’s and Gustafson’s speedup laws for a sequential fraction of 10%.

Chapter 4

Pipelining Principles and Superscalar Design

Pipelining is a primary instruction-level parallelism technique that overlaps instruction stages in hardware. This chapter details linear and non-linear pipeline architectures, analyzes execution hazards, and demonstrates how to optimize non-linear scheduling.

4.1 A 5-Stage Linear Instruction Pipeline

Modern instructions are typically executed in multiple physical stages, separated by electrical barriers called interface latches.

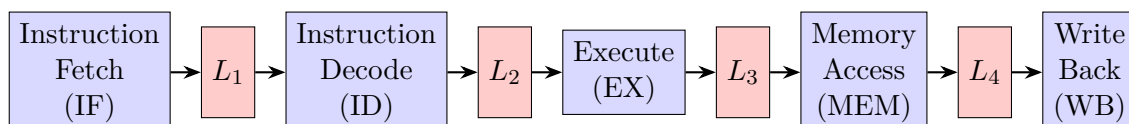


Figure 4.1: A classic 5-stage linear instruction pipeline with stabilizing interface latches (L_1 to L_4).

The interface latches (L_1, L_2, L_3, L_4) act as barriers that hold the computed results of the current clock cycle, allowing the next stage to work on them during the subsequent clock cycle without corruption.

4.1.1 Pipeline Space-Time Diagram

To visualize how instructions flow through a pipeline over time, we use a **Space-Time Diagram**.

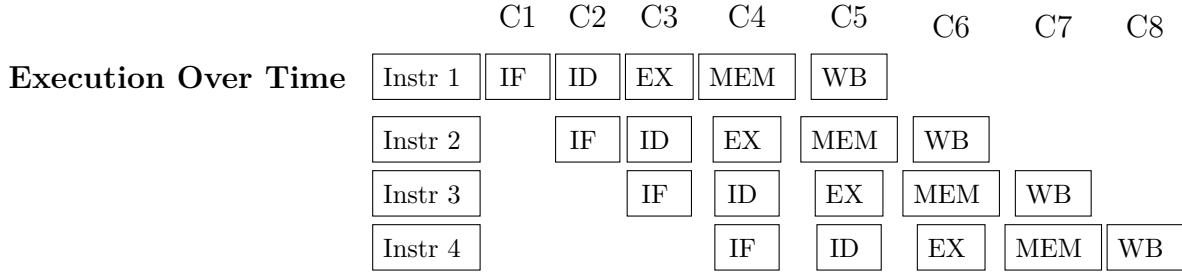


Figure 4.2: Space-Time / Execution diagram of a 5-stage pipeline processing four instructions.

4.2 Non-Linear Pipeline Scheduling

In a linear pipeline, data flows forward from one stage to the next. In a **non-linear pipeline**, hardware units can be reused for different tasks at different times during an instruction's execution. This introduces feedback loops and feed-forward bypass paths, which can lead to stage collisions.

4.2.1 Reservation Tables

We use a **Reservation Table** to map stage usage over time. The rows represent physical pipeline stages, and the columns represent clock cycles.

Stage	t_1	t_2	t_3	t_4
S_1	X			X
S_2		X		
S_3			X	

Figure 4.3: Sample non-linear pipeline Reservation Table.

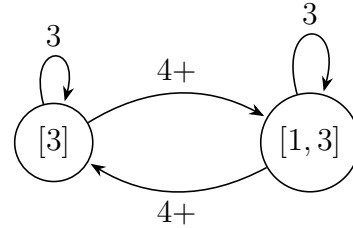


Figure 4.4: State Transition Diagram.

4.2.2 Deriving Schedules and Collision Vectors

Using the reservation table in Figure 4.3, let us calculate how to schedule tasks without collisions:

1. **Forbidden Latencies:** These are the number of clock cycles between any two 'X' marks in the same row. In stage S_1 , there are 'X' marks at t_1 and t_4 . The difference is $4 - 1 = 3$. Therefore, the forbidden latency is 3.
2. **Collision Vector:** We construct a binary vector $C = (c_m c_{m-1} \dots c_1)$ where $c_i = 1$ if i is a forbidden latency, and 0 otherwise. For our table, the collision vector is $C = (100)$ (since only 3 is forbidden).

3. **State Transition Analysis (Figure 4.4):** We track states using the active collision vector. By analyzing the loops (cycles) in this state machine, we identify collision-free schedules.
4. **Greedy Cycles and MAL:** A greedy cycle is a path through the state diagram where we always choose the lowest possible latency transition. The **Minimum Average Latency (MAL)** is the average latency of the optimal cycle, representing the maximum throughput the pipeline can sustain.

Chapter 5

Shared-Memory Consistency and Cache Coherence

In multi-core systems, each processor has its own private, high-speed cache. This layout introduces the cache coherence problem: ensuring that all caches maintain a consistent, unified view of shared memory. This chapter covers memory consistency models, cache mapping, and coherence protocols.

5.1 Memory Consistency Models

When multiple processors read and write to shared memory, we need rules to define the exact order in which memory updates become visible to other processors. These rules are called **memory consistency models**.

5.1.1 Sequential Consistency (SC)

Defined by Leslie Lamport in 1979, Sequential Consistency is the most intuitive memory model.

Definition 5.1 (Sequential Consistency). *A system is sequentially consistent if the result of any execution is the same as if the operations of all processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.*

To visualize this, imagine an execution arbiter that selects one instruction from any processor, executes it on memory, and then selects the next, like a rotating switch. No processor's memory operations can be reordered out of program order. While SC is easy to understand, it prevents many hardware optimizations (such as out-of-order execution and write buffering), resulting in lower system performance.

5.1.2 Weak Consistency (WC)

Weak consistency relaxes these rules. It splits memory operations into ordinary data accesses (loads/stores) and synchronization operations (such as acquiring or releasing a lock). It guarantees that:

1. Synchronization operations are sequentially consistent.
2. No ordinary memory access is allowed to pass a synchronization operation.
3. Ordinary reads and writes can be aggressively reordered between synchronization points to maximize throughput.

5.2 Cache Mapping Architectures

When data is loaded from main memory into the cache, we use a mapping scheme to determine where it is placed. In an **N-Way Set Associative** cache, the cache is divided into sets, and each memory address maps to a specific set but can be placed in any of the N ways (lines) within that set.

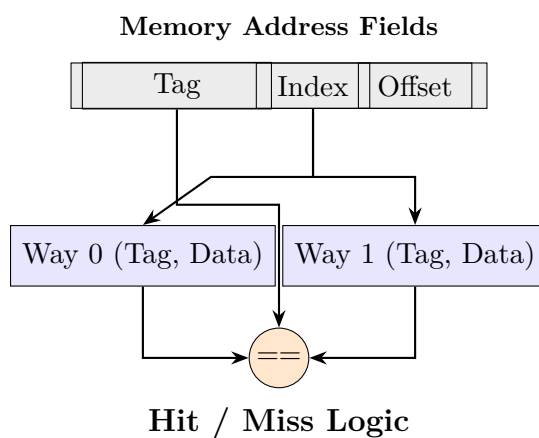


Figure 5.1: Hardware block diagram of a 2-way Set Associative Cache mapping logic.

The index bits of the memory address are used to select a target set. The tag bits are then compared in parallel with the tags in both Way 0 and Way 1. If a match is found and the line is valid, a cache hit occurs.

5.3 The MESI Cache Coherence Protocol

The **MESI Protocol** is a common protocol used to maintain cache coherence. Every cache line is tracked in one of four states:

- **Modified (M):** The cache line is present only in the current cache and is dirty (newer than the copy in main memory). The processor has exclusive permission to write to this line.
- **Exclusive (E):** The cache line is present only in the current cache but is clean (matches main memory).
- **Shared (S):** The cache line is present in the current cache and potentially other caches. It is clean and read-only.
- **Invalid (I):** The cache line does not contain valid data. Reading from it triggers a cache miss.

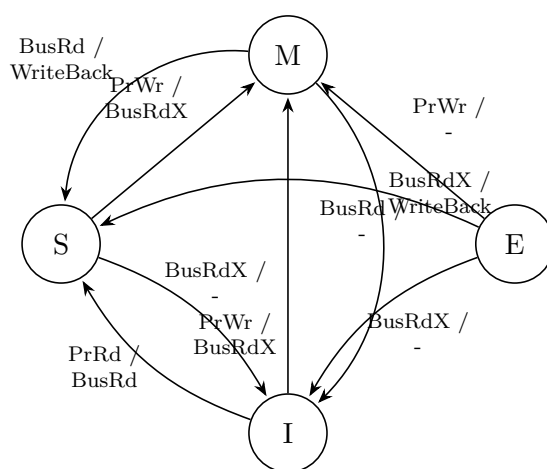


Figure 5.2: MESI Cache Coherence Protocol state machine showing transition triggers (Processor Read/Write, Bus Read/Read-With-Intent-To-Modify) and resulting actions.

The state machine in Figure 5.2 handles local processor read and write events alongside external bus transactions observed (snooped) from other processors on the system bus.

Chapter 6

Vector and SIMD Architectures

Applications involving massive data parallelism (such as scientific simulations and neural network training) are highly repetitive. This chapter details hardware structures tailored for uniform vector operations.

6.1 Vector Processing Principles

Vector processors operate on entire arrays of data (vectors) using a single instruction. This reduces instruction fetch and decode overhead compared to traditional scalar loop execution.

6.1.1 Register-to-Register Vector Pipeline Execution Flow

In a Register-to-Register vector architecture, vector operands are first loaded from memory into large, multi-element vector registers. These registers then feed data directly into pipelined functional units, which write the results back to another vector register.

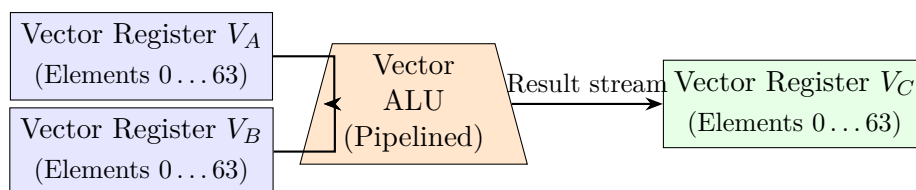


Figure 6.1: Data flow in a Register-to-Register vector execution pipeline.

Because the system knows it will process a long sequence of elements in order, it can stream them through the pipelined ALU without incurring instruction-fetch overhead for each individual element.

6.2 The Connection Machine CM-5

The Thinking Machines Connection Machine CM-5 is a historically significant supercomputer that combined aspects of both SIMD and MIMD designs.

Architecturally, the CM-5 was composed of hundreds to thousands of processing nodes, each equipped with custom vector arithmetic units. These nodes were coordinated by a control network and connected by a high-bandwidth data network.

The CM-5 organized its nodes using a **Fat-Tree** topology. Unlike a standard tree topology, where connections near the top (root) can become communication bottlenecks, a Fat-Tree increases the bandwidth (the “thickness” of the links) as you move up the hierarchy toward the root. This structure allows the system to sustain high global communication rates across thousands of individual processing nodes.

Conclusion

The evolution of computer architecture is a continuous balance between physical hardware constraints and software demands. From the early models of parallelism to modern highly parallel vector units, the primary goal has remained the same: to execute instructions as efficiently and concurrently as possible.

By mastering these fundamentals—the mathematical limits of speedup, data dependencies, memory consistency models, pipelining, and cache coherence—engineers can design systems that continue to scale performance long after the physical boundaries of individual transistors have been reached.